Rapport RAPPORT DE LABORATOIRE PRATIQUE

Mini-RAG & Local AI with Ollama

Module : M1 Agentic AI & Building AI Agents

Réalisé par : IKRAM LEFHAL LALAOUI

Encadré par : Pr. Lotfi EL AACHAK et Pr. Ikram Benabdelouahab

Année universitaire 2026–2027

Sommaire

1. **Introduction and Objectives.....3**
2. **Experimental Evaluation: Model Benchmarking……..3**
3. **System Architecture and Implementation: FSTT Local AI Assistant.4**
4. **Security and Privacy Considerations……..6**
5. **Conclusion 7**

1. Introduction and Objectives

The Doctoral Training Module FDO – M1 Agentic AI & Building AI Agents provides a foundation upon which to develop fully autonomous multi-agent systems. Therefore, before designing agents that reason, call tools, make decisions etc. It is essential to have a robust local inference architecture. Cloud based APIs (OpenAi etc.) can be highly performant however, they present significant challenges to scientific researchers: Data Privacy Risks; Per Token Costs; Network Dependency. This Lab explores Local AI utilizing the Ollama Framework providing the capability to execute large language models locally to provide data sovereignty and off-line functionality.

A key objective of this hands-on work is to connect theoretical knowledge with practical engineering skills by building a full end-to-end pipeline including: Deploying quantized local models (qwen3.5 and gemma3); Interfacing with a local inference server via REST API using Python; State Management: Implementing conversational memory to resolve the problem of the inherent statelessness of standard LLM API calls. Additionally, the Lab will explore Retrieval-Augmented Generation (RAG): By implementing a custom retrieval pipeline (TF-IDF) to ground LLM generated responses against a collection of local textual documents, we will reduce hallucination and increase the level of factual accuracy. All components will be combined within the FSTT Local AI Assistant, a Full Stack Web Application developed using Flask. This application presents not only the deployment of the local LLM and RAG Pipeline but also Structured JSON Outputs and Real-Time Performance Monitoring (Tokens/S and Latency). Ultimately, the FSTT Local AI Assistant represents the culmination of an Empirical Benchmark investigating the Trade-Offs Between Model Size, Inference Speed and Memory Footprint.

2. Experimental Evaluation: Model Benchmarking

The To evaluate the viability of local AI deployment for agentic workflows, an empirical benchmark was conducted comparing two local models: qwen3.5 (6.55 GB) and gemma3 (3.35 GB). The evaluation focused on three critical metrics: memory footprint (VRAM), inference speed (tokens/second), and total response latency.

2.1. Empirical Results

The experimental data reveals a clear and expected trade-off between model size and performance. The results are summarized in the table and figure below:

	Model	Disk size (GB)	Loaded size (GB)	VRAM (GB)	Load time (s)	Response time (s)	Tokens	Tokens/s	System RAM used (%)	Quality /5
0	qwen3.5	6.55	5.61	5.61	32.33	177.62	3283	22.66	20.7	None
1	gemma3	3.35	2.88	2.88	14.22	44.32	837	47.53	25.7	None

Table 1: Empirical Benchmark Results of Qwen 3.5 and Gemma 3.

Metrics measured include disk size, VRAM usage, load time, response time, and generation speed (tokens/s)

2.2. Visual Analysis of the Size-Performance Trade-off

To better visualize these metrics, the following charts illustrate the comparison across three dimensions: generation speed, memory consumption, and total latency.

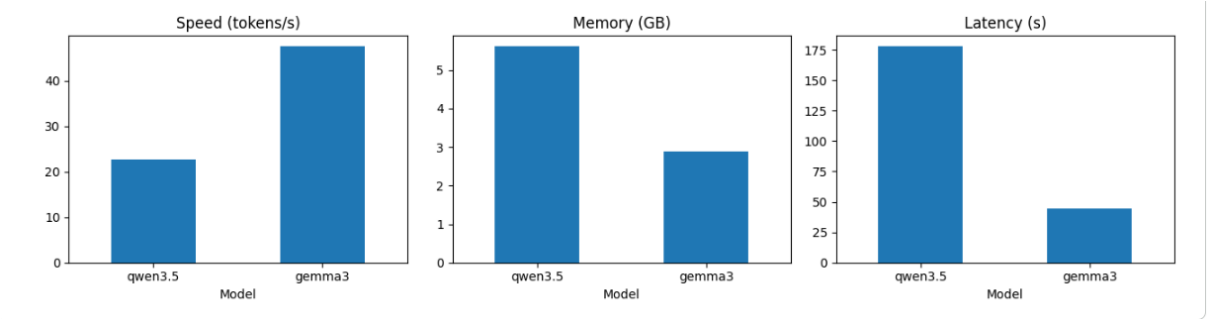


Figure 1: Performance Metrics Comparison between Qwen 3.5 and Gemma 3.

The bar charts display (left) Generation Speed in tokens/s, (middle) Loaded Memory in GB, and (right) Total Response Latency in seconds.

2.3. Analysis of the Results

The measurements demonstrate that the larger model, qwen3.5, requires significantly more hardware resources, consuming 5.61 GB of VRAM and taking 32.33 seconds to load from disk. Conversely, the smaller gemma3 model operates with a much smaller footprint, using only 2.88 GB of VRAM and loading in 14.22 seconds.

In terms of inference speed, gemma3 proved to be substantially faster, generating 47.83 tokens per second, compared to 22.86 tokens/s for qwen3.5. Because qwen3.5 generated a much longer response (3,283 tokens vs. 837 tokens), its total latency reached 177.62 seconds.

2.4. Deployment Implications for Agentic AI

This benchmark highlights a crucial consideration for building AI agents. While larger models like qwen3.5 may offer deeper reasoning capabilities, their high latency and memory requirements make them less suitable for real-time, multi-agent systems where rapid tool-calling and decision loops are required. For resource-constrained environments or edge devices, smaller models like gemma3 provide a highly efficient alternative, ensuring fast inference and low memory usage, which is essential for scalable local deployments.

3. System Architecture and Implementation: FSTT Local AI Assistant

The culmination of this laboratory was the development of the FSTT Local AI Assistant, a web-based application designed to demonstrate the practical integration of local LLMs, conversational memory, and Retrieval-Augmented Generation (RAG). The system was built using Python and the Flask web framework, serving as a lightweight, local alternative to cloud-based AI interfaces.

3.1. Architectural Overview

The application follows a client-server architecture that decouples the user interface from the inference engine. The architecture consists of three main layers:

- Frontend (Client): An HTML/JavaScript interface that captures user queries and displays the AI's responses, including real-time metrics (latency, tokens per second) and the specific documents retrieved.
- Backend (Flask Server): A Python server that handles HTTP requests, manages conversation history, coordinates the RAG retrieval, and communicates with the LLM.
- Inference & Retrieval Layer: The local Ollama server running the qwen3.5 model, combined with a custom TF-IDF Retriever that indexes the local .txt documents.

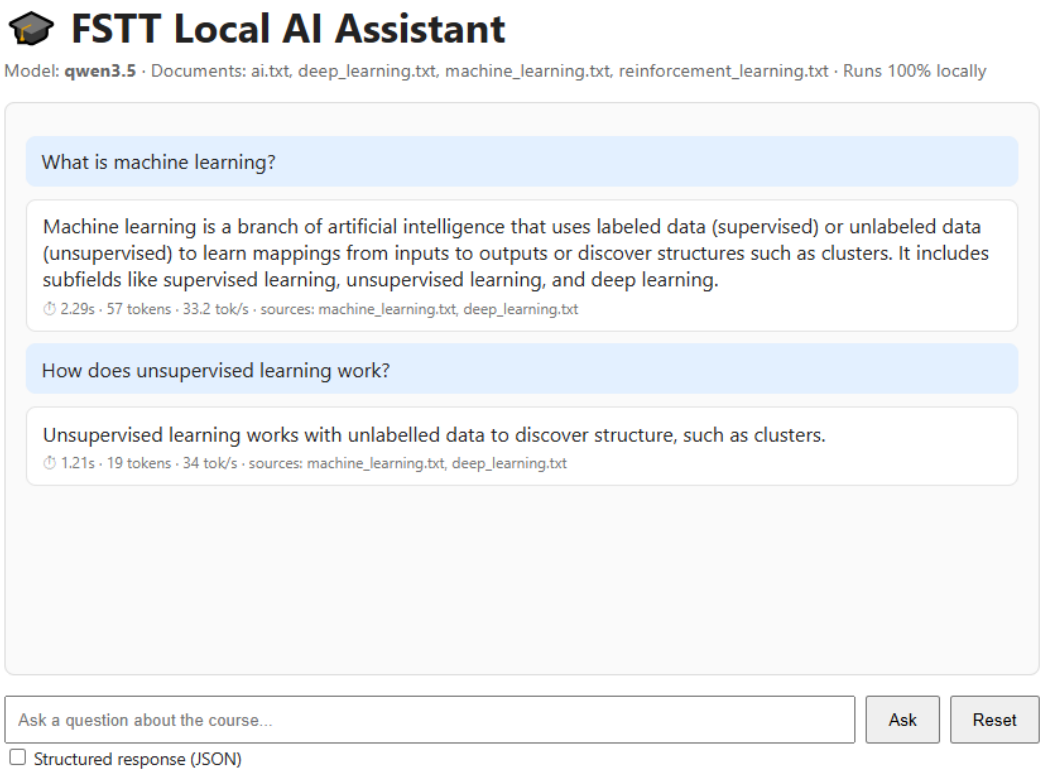


Figure 2: User Interface of the FSTT Local AI Assistant.

The interface allows users to query local documents, with an option to enforce structured JSON outputs.

3.2. The RAG Pipeline Implementation

To prevent the LLM from hallucinating and ensure it remains grounded in factual data, a custom RAG pipeline was implemented:

- **Ingestion:** The system loads multiple .txt documents (e.g., deep_learning.txt, reinforcement_learning.txt) from a local directory.
- **Retrieval (TF-IDF):** When a user submits a query, the backend uses a custom Term Frequency-Inverse Document Frequency (TF-IDF) algorithm to calculate the cosine

similarity between the query and the local documents. It extracts the top-k most relevant chunks.

- **Context Injection:** The retrieved text chunks are injected into the system prompt as CONTEXT. The LLM is explicitly instructed: *"Answer the question using only the CONTEXT below. If the answer is not contained in the context, say 'Information not available'."*
- **Structured Output:** For specific queries, the system leverages Ollama's structured output capabilities, forcing the LLM to return a strictly formatted JSON object (containing keys like topic, summary, key_points, and difficulty). This is critical for downstream automation in agentic workflows.

3.3. State Management and Conversational Memory

Because LLMs are stateless (they do not remember previous interactions), a conversational memory system was implemented. The Flask backend maintains a history list, storing alternating user and assistant messages. Before querying the LLM, the backend injects the last N*N* turns of the conversation into the prompt.

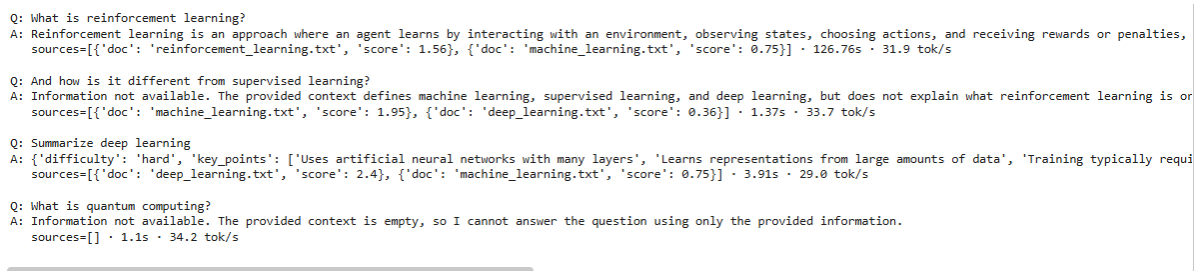


Figure 3: Retrieval-Augmented Generation in Action.

The system correctly identifies and retrieves deep_learning.txt to answer the user's query, while declining to answer out-of-domain questions.

4. Security and Privacy Considerations

Deploying local LLMs introduces unique security challenges that must be addressed, especially in environments where sensitive research or corporate data is processed. By default, the Ollama server binds to 127.0.0.1 (localhost), ensuring that only applications running on the same machine can access the model. However, if a researcher modifies the environment variable to OLLAMA_HOST=0.0.0.0 to allow network access, the server becomes exposed to the local network without any form of authentication.

In an enterprise or highly secure environment (such **as** Kaspersky), this would constitute a critical vulnerability. An unauthenticated LLM server allows any actor on the network to hijack the model for unauthorized compute (leading to Denial of Service) or, in the case of a RAG system, exploit prompt injection to extract sensitive data from indexed documents. To secure such deployments, it is mandatory to place the inference server behind a reverse proxy (e.g., Nginx) enforcing TLS encryption, API-key authentication, and strict firewall rules.

5. Conclusion

This laboratory successfully demonstrated the end-to-end deployment of a Local AI infrastructure using Ollama, serving as a robust foundation for building future autonomous AI agents. By moving away from cloud-based APIs, we achieved complete data sovereignty, zero inference costs, and offline functionality.

The implementation of the FSTT Local AI Assistant proved that local models can be integrated into functional, full-stack web applications. The integration of a custom TF-IDF Retriever for RAG successfully eliminated hallucinations, ensuring that the AI remained strictly grounded in the provided academic documents. Furthermore, the empirical benchmark highlighted the crucial trade-off between model size and speed, proving that smaller models (like Gemma 3) are highly viable for resource-constrained environments, whereas larger models (like Qwen 3.5) are better suited for complex, asynchronous reasoning tasks. Ultimately, this practical work bridged the gap between theoretical LLM concepts and the applied engineering required to develop secure, private, and autonomous agentic workflows.